

Relatório do EP1 de MAC0425 - Inteligência Artificial

Laufernando Souza Dias - 11222947

30 de março de 2026

a - compilado em tabelas das estatísticas das buscas implementadas em termos de nós expandidos e tamanho de plano;

Tabela 1: Resultados das buscas - Cenário 1 (PositionSearchProblem)

Algoritmo	Mapa	Custo do Plano	Nós Expandidos
DFS	tinyMaze	10	14
DFS	mediumMaze	130	144
DFS	bigMaze	210	390
BFS	mediumMaze	68	269
BFS	bigMaze	210	620
UCS	mediumMaze	68	269
StayEast (UCS)	mediumDottedMaze	1	186
StayWest (UCS)	mediumScaryMaze	68719479864	108
A* (Manhattan)	bigMaze	210	549
IDDFS	mediumMaze	68	15429
IDDFS	bigMaze	210	60312

Tabela 2: Resultados das buscas - Cenário 2 (CornersProblem)

Algoritmo	Mapa	Custo do Plano	Nós Expandidos
BFS	tinyCorners	28	252
BFS	mediumCorners	106	1966
A* (Corners Heuristic)	mediumCorners	106	692

b - discussão das vantagens e desvantagens de cada método;

Cenário 1

Ao compararmos a DFS com a BFS, confirmou-se as principais diferenças citadas entre as duas: a BFS ganha em otimalidade (pelas diferenças dos resultados em *mediumMaze*), enquanto a DFS tem complexidade de espaço menor e expandiu menos nós para os mesmos labirintos ($O(b \times m)$ contra $O(b^{d+1})$, onde b é o grau de ramificação, m a profundidade máxima, e d é a profundidade da solução mais curta).

Há equivalência entre BFS e UCS em *mediumMaze* (também esperado, já que nesse teste a UCS é o caso especial em que funciona como BFS, pela ausência de "arestas" mais longas, recompensas ou obstáculos). Já o comportamento da UCS para os outros labirintos é abordado na Questão 3.

Por fim, A* e a IDDFS (aprofundamento iterativo) tiveram soluções ótimas, como devem ser em casos de custo uniforme, mas o ponto fraco da IDDFS ficou evidenciado: muitos nós expandidos ao "ir e voltar" na busca para diferentes profundidades.

Cenário 2

A notável diferença foi a quantidade de nós expandidos em A* com a heurística implementada (tentar ir para o canto mais próximo, depois para aquele mais próximo do canto atual, e assim sucessivamente): quase três vezes menor que na BFS, mostrando uma vantagem de se ter uma busca informada frente a uma busca cega.

c - questões teóricas - respostas;

Questão 1

Sim. No caso da DFS, é utilizada uma pilha para guardar os sucessores e tentar a expansão. Como consequência, nós para oeste são explorados primeiro, e isso se reflete numa busca como a da imagem: expansão para oeste, solução não ótima foi encontrada primeiro, mas de acordo com a ordem de exploração:

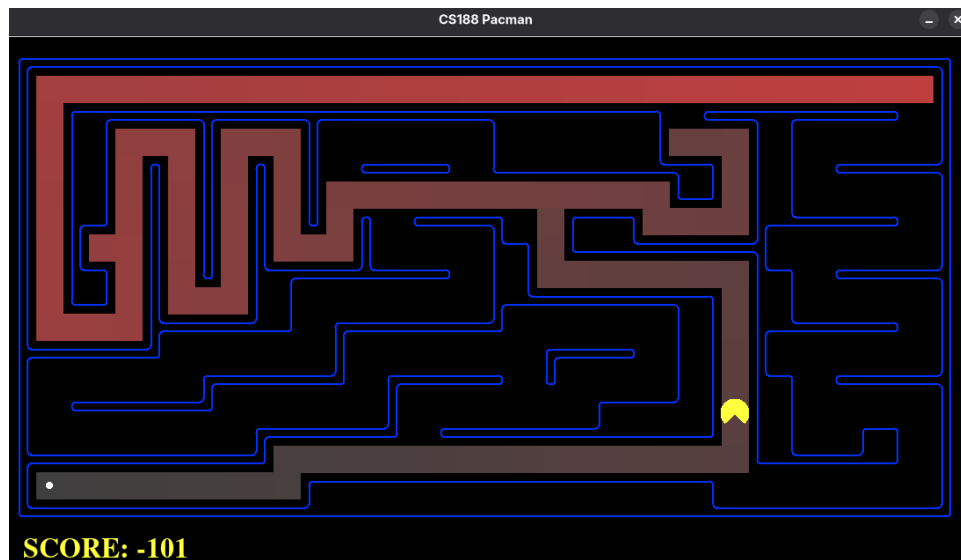


Figura 1: Expansão de nós na DFS

A busca em grafo com a memorização de nós visitados também é importante para evitar cair em ciclos e expansão repetida de mesmos estados, ao compararmos com busca em árvore.

Questão 2

Sim. Alterar a ordem de geração de estados sucessores não remove otimalidade da BFS, pois esta expande os nós garantindo que encontrará uma solução de menor profundidade, e consequentemente menor custo possível em casos de grafo com custo uniforme.

Questão 3

Em *mediumDottedMaze*, o agente sofre pequenas penalizações em posições à oeste, então movimentações para a esquerda são compensadas por haver muita comida no caminho ótimo encontrado.

Já em *mediumScaryMaze* as penalizações em posições à leste são grandes e isso gera um custo total alto, mesmo que seja ótimo - pois alguns movimentos para a direita são necessários (a classe *StayWestSearchAgent* deve ter sido implementada assim para simular o lado direito como perigoso e cheio de fantasmas sem ter que lidar com eles diretamente, como é mostrado graficamente na interface aberta):

Questão 4

O algoritmo A* encontra solução mais rapidamente porque a heurística guia a busca em direção ao objetivo, reduzindo o número de nós que precisam ser expandidos. Quanto melhor (mais informada) a heurística, menor tende a ser o espaço de busca explorado. Por exemplo, se modificarmos a heurística implementada em *cornersHeuristic* para que devolva sempre zero (heurística trivial), o A* se comporta como a busca de custo uniforme (UCS), expandindo um número significativamente maior de nós, semelhante ao observado na BFS em problemas com custo uniforme. Isso evidencia que a heurística é fundamental para o ganho de eficiência.

Uma heurística consistente implica que, para todo estado n e sucessor n' , vale que:

$$h(n) \leq c(n, n') + h(n')$$

ou seja, ela respeita a desigualdade triangular. Isso garante que os valores de $f(n) = g(n) + h(n)$ são não decrescentes ao longo de um caminho.

No caso do A*, utilizar uma heurística consistente traz duas vantagens principais: (i) ela é necessariamente admissível, garantindo que a solução encontrada é ótima; e (ii) evita a necessidade de reavaliar ou reexpandir nós já explorados, pois o primeiro caminho encontrado para um estado já é o de menor custo. Isso torna a implementação mais eficiente e simplifica o controle da fronteira.

No problema considerado, utiliza-se, por exemplo, a distância de Manhattan como heurística, que é consistente porque nunca superestima o custo real até o objetivo e respeita a estrutura do espaço de estados (movimentos em grade). Dessa forma, o A* consegue direcionar a busca de forma eficiente, expandindo menos nós do que algoritmos não informados como BFS ou UCS.

d - possíveis melhorias e dificuldades;

Apesar do sistema apresentar uma boa estrutura modular para experimentação com algoritmos de busca, algumas melhorias podem ser propostas.

Primeiramente, seria interessante desacoplar ainda mais a lógica de busca da representação do problema. Embora a interface *SearchProblem* já abstraia bem os métodos principais, a forma como os estados e caminhos são manipulados dentro dos algoritmos ainda exige atenção manual, o que pode de início levar a confusões e erros de implementação. Uma padronização mais rígida da estrutura dos estados (por exemplo, uso de objetos ou tuplas nomeadas) poderia reduzir ambiguidades.

Além disso, o sistema poderia oferecer mecanismos de visualização mais ricos da busca, como diferenciação entre nós explorados, fronteira e caminho final, ou até animações que mostrem a evolução da fronteira ao longo do tempo. Apesar de já ser possível presumir como foram feitas algumas das expansões, isso facilitaria a compreensão do comportamento dos algoritmos, especialmente em buscas mais complexas como A* e IDDFS e em labirintos maiores como o *bigMaze*.

Já a implementação da IDDFS em si foi feita usando recursão, e certamente poderia receber otimizações a fim de potencializar seu uso sem expansões excessivas.

Uma das principais dificuldades encontradas durante o desenvolvimento foi compreender corretamente a interface orientada a objetos utilizada no projeto, especialmente no que diz respeito à interação entre os algoritmos de busca e a classe *SearchProblem*. Em particular, o entendimento da estrutura retornada pelo método *getSuccessors* — uma tripla contendo estado sucessor, ação e custo — foi fundamental para evitar erros de execução.

Além disso, o uso adequado das estruturas de dados fornecidas, principalmente fila de prioridades, exigiu atenção, principalmente no caso do algoritmo de custo uniforme e do A*, onde a atualização de prioridades (método *update*) precisa ser feita corretamente para evitar comportamento incorreto ou loops.

Por fim, a implementação e entendimento de heurísticas consistentes no A* exigiram maior esforço conceitual, especialmente para garantir que a heurística fosse ao mesmo tempo admissível e eficiente, influenciando diretamente o desempenho do algoritmo.